Lab Report

CI/CD with Render: Docker, Managed PostgreSQL, Staging & Process Lifecycle
Mood Journal — A New Application Taken Through a Full Delivery Pipeline

Achindra Sharma — 2547105 (Individual Submission)

4MCA A — MCA Trimester 5: DevOps Lab

Repository: github.com/Achindra2003/mood-journal

Production: https://mood-journal-drkn.onrender.com

Staging: https://mood-journal-stage.onrender.com

CI/CD Status: **GitHub Actions Run #41805271089 (main, merge of PR #1) — Passed**

1. Project Description & System Context

Mood Journal is a small Node.js/Express application written for this lab. A user picks a mood from 1 to 5, adds an optional note and sees a list of past entries that can be deleted. The application is deliberately simple: the purpose of the lab is the delivery pipeline around it, not the features. The pipeline covers a new Git repository, a Docker image, GitHub Actions CI, deployment to Render, a managed PostgreSQL database, a separate staging environment, promotion to production through a pull request, and correct behaviour of the process under signals and failures.

The application was first built on SQLite so that the problem with ephemeral storage on Render's free tier could be demonstrated, and then migrated to PostgreSQL so that the fix could be demonstrated against the same workflow.

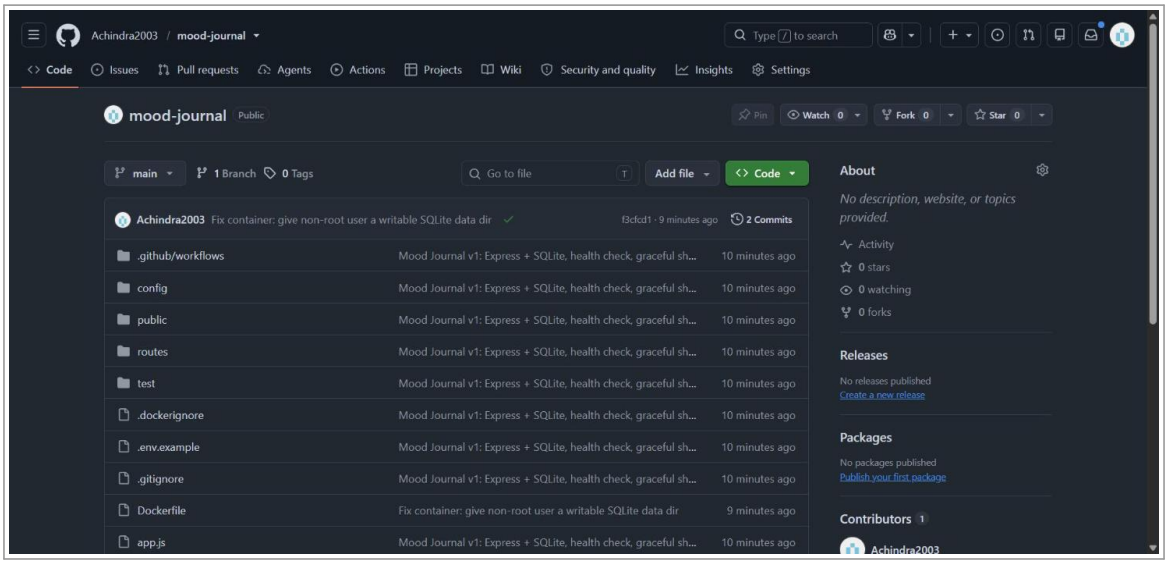


Figure 1 — The new public repository Achindra2003/mood-journal on GitHub, created from the command line.

2. Repository, Docker Image & Continuous Integration

The repository contains the Express application, a single-stage Dockerfile based on `node:22-alpine`, and a GitHub Actions workflow that runs on every push and pull request to `main` and `stage`. The workflow installs dependencies, runs the automated tests, builds the Docker image and starts the container to smoke-test the `/health` endpoint.

The very first CI run **failed**, and the failure was a real defect rather than a configuration slip. The image runs as the unprivileged `node` user, which cannot write to `/app`, so SQLite could not create its database file and the container exited during startup. The fix was to create a data directory owned by that user. The second run passed. The failed run is kept in the evidence because it shows the smoke test doing its job.

```
Failed to initialize database: Error: unable to open database file
  code: 'ERR_SQLITE_ERROR', errstr: 'unable to open database file'
Process completed with exit code 1.        (CI #1, commit 02352c8)

Fix: RUN mkdir -p /app/data && chown -R node:node /app/data   -> CI #2 (commit f3cfcd1) passed
```

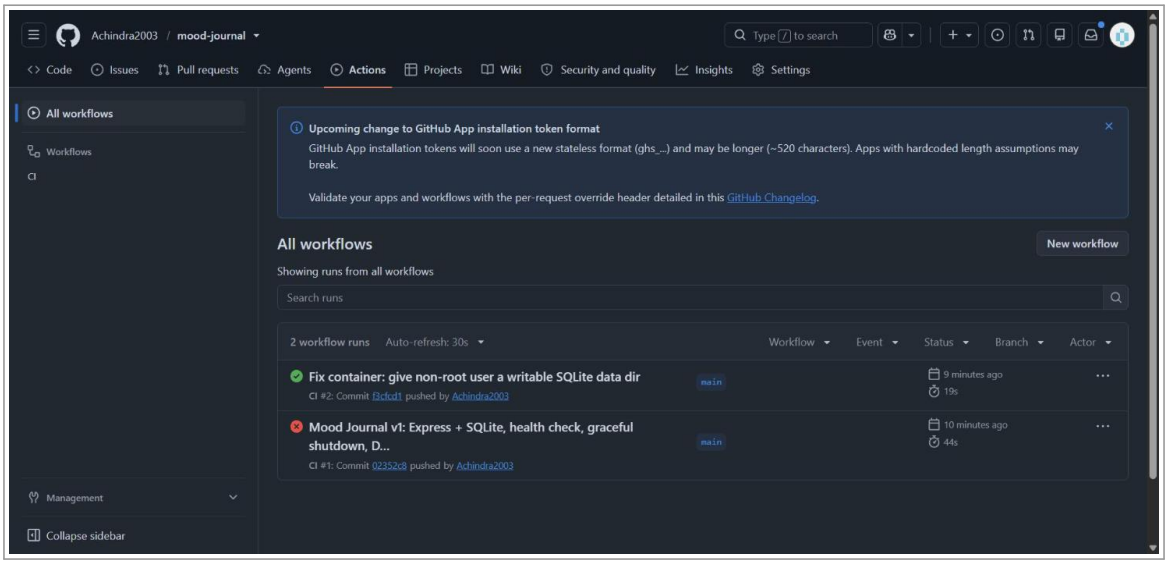


Figure 2 — GitHub Actions history: CI #1 failed (container could not open its SQLite file), CI #2 passed after the fix.

3. Deployment to Render & the Production Domain (Steps 1 and 2)

A Render web service **mood-journal** was created on the Free plan with the Docker runtime, connected to the repository's `main` branch with auto-deploy on every commit and health check path `/health`. The service was created with the Render CLI.

A tooling pitfall. The CLI was run from Git Bash, which rewrote the argument `/health` into `C:/Program Files/Git/health` before Render received it. The first deploy therefore never passed its health check and sat in *In progress* until it was cancelled. The path was corrected from PowerShell and a new deploy went live. This is recorded under challenges as well.

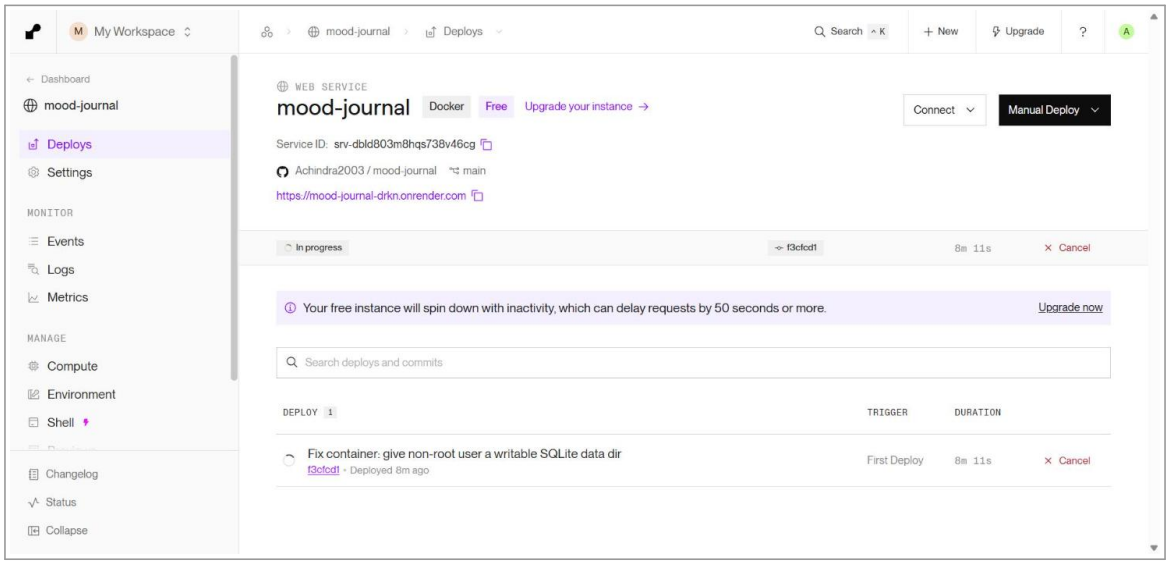


Figure 3 — The Render service mood-journal (Docker, Free) building its first deploy from the main branch.

With the service live, the production domain was opened in a browser and an entry was added through the form (mood 4, note “First entry on the production domain”). Every later commit to `main` (README, screenshots, the migration merge) triggered an automatic deploy with no manual step.

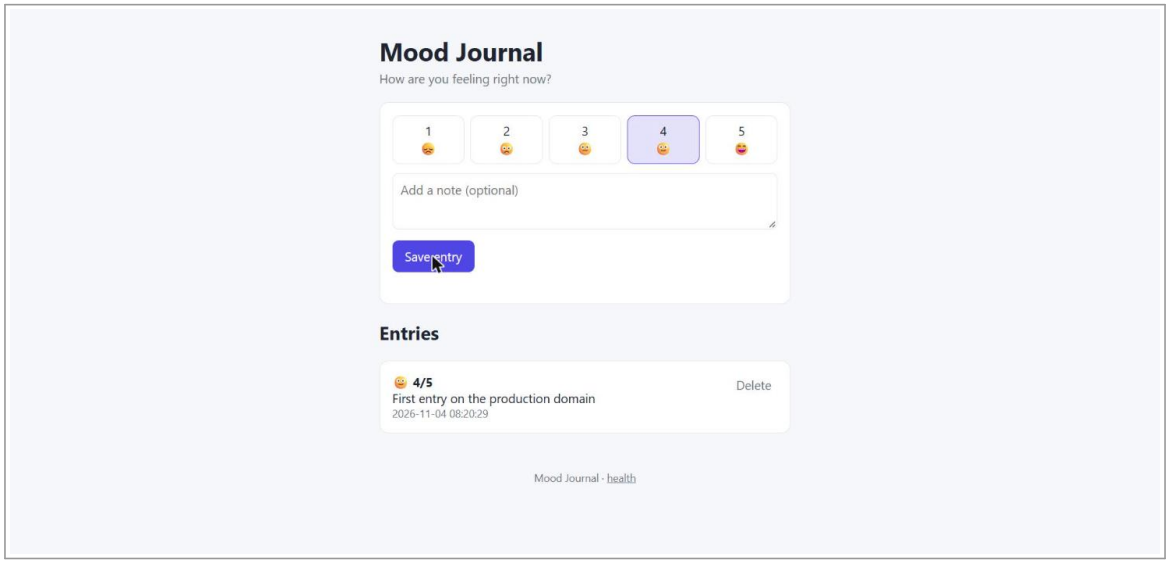


Figure 4 — The production domain mood-journal-drkn.onrender.com with a new entry saved through the browser (SQLite version).

4. Demonstrating Ephemeral Storage

Render's free web services keep their filesystem inside the container image, so anything written at runtime is lost when the instance is replaced. To show this, the same commit was redeployed with a manual deploy while the application was still using SQLite. After the redeploy the entry added in the previous section was gone and the API returned an empty list.

```
$ render deploys create srv-dbld803m8hqs738v46cg        # redeploy of commit dd1bff6
Live   dd1bff6   api   2026-11-04T08:20:45Z -> 08:21:08Z

$ curl https://mood-journal-drkn.onrender.com/api/moods
[]                                                  <- the entry was lost
```

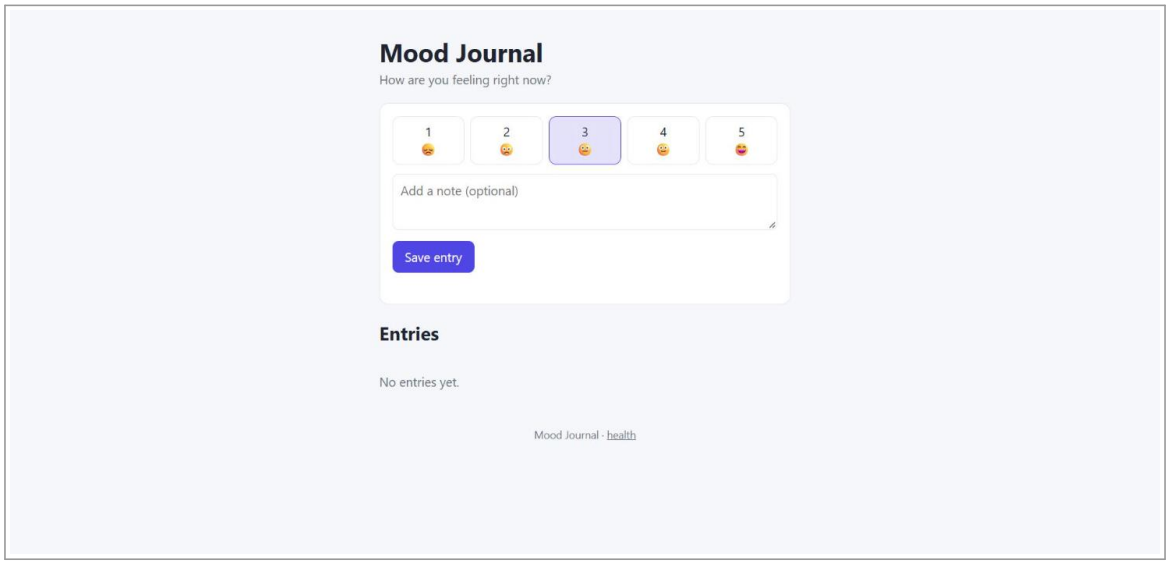


Figure 5 — The same production site after redeploying the same commit: "No entries yet." — the SQLite data did not survive.

5. Managed PostgreSQL & Local Development (Steps 3 and 4)

The free Render plan allows one PostgreSQL instance per account, and the existing instance **dream-catcher-db** (from the previous lab) already used it. Rather than delete that data, two separate *databases* were created inside the same instance: **moods_prod** for production and **moods_stage** for staging. Production and staging therefore never share rows, although they share one database server. A dedicated instance per environment would be the better arrangement on a paid plan.

The application was migrated from SQLite to the `pg` driver on a `stage` branch. The changes were: a connection pool built from `DATABASE_URL` (with TLS for Render, none for local), `SERIAL` ids and `TIMESTAMPTZ DEFAULT now()`, parameterised queries (`$1`, `RETURNING *`), a `/health` endpoint that executes `SELECT 1`, an error-handling middleware, a `docker-compose.yml` with a local Postgres, and a Postgres service container in the CI workflow.

The automated tests (6 in total: input validation, health, create, list and delete) ran against a real PostgreSQL database and passed. The application was then started locally against **moods_stage** and an entry was added through the browser at `localhost:3001`.

```
$ npm test
# tests 6
# pass 6
# fail 0

$ npm start
Server running on http://localhost:3001
```

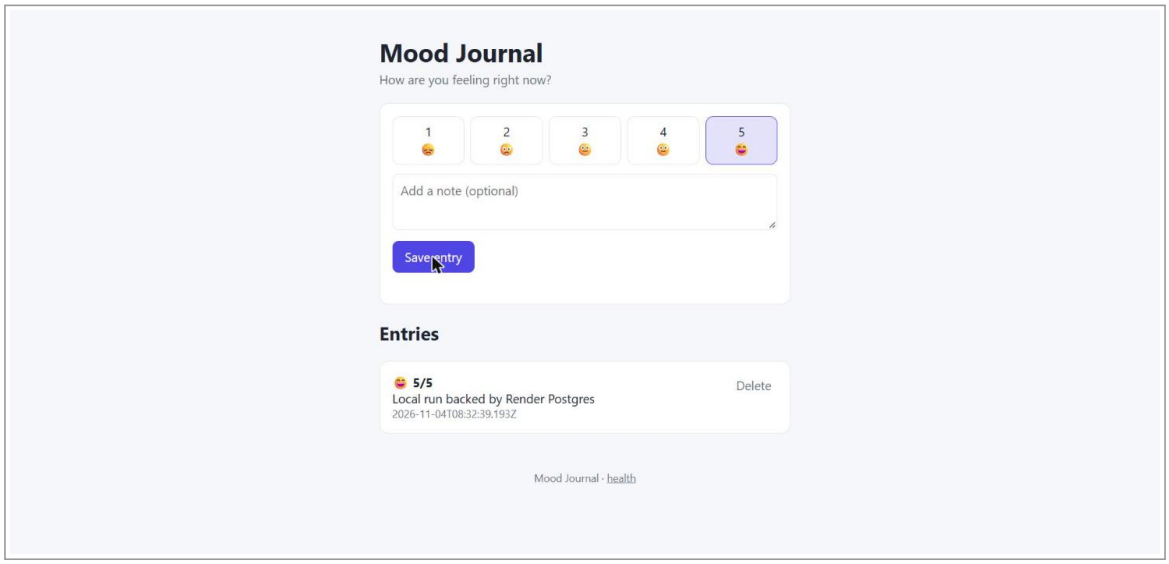


Figure 6 — Local run on localhost:3001: an entry added through the UI and stored in the Render PostgreSQL database moods_stage.

The files were edited directly in the project source and committed with Git; no claim is made here about which editor was used.

6. Staging Environment: the `stage` Branch (Step 5)

A second Render web service, **mood-journal-stage** (Docker, Free plan), was created and pointed at the `stage` branch. It has its own environment variables and its own database (moods_stage). CI ran on the branch push and passed.

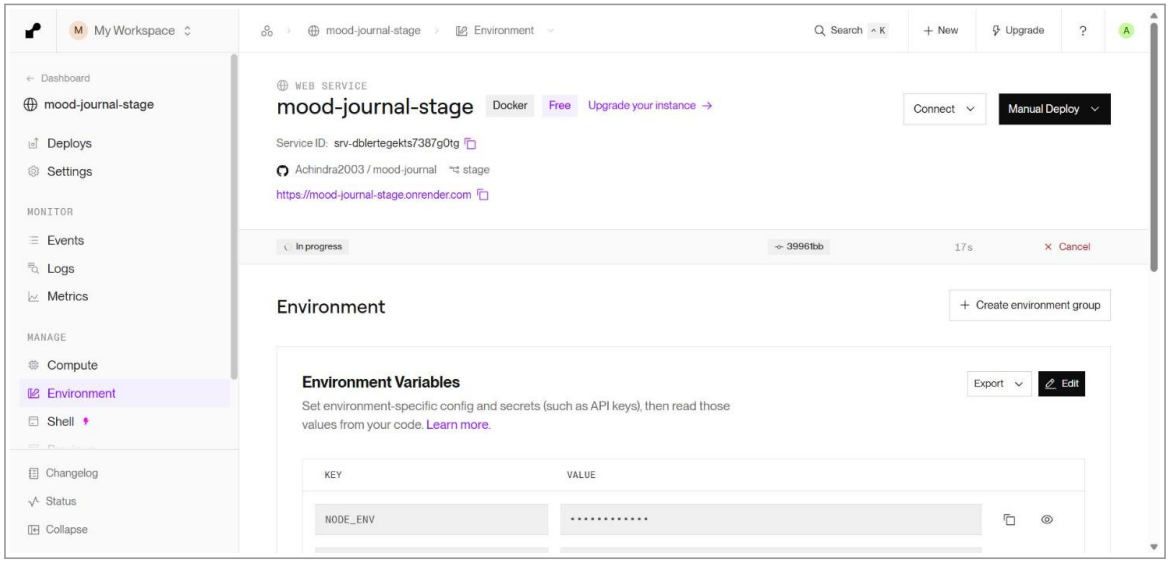


Figure 7 — The separate staging web service mood-journal-stage, tracking the stage branch (Free plan).

The secret `DATABASE_URL` was entered into the Render dashboard by hand, not by the automation, so the stage service's first deploy **failed**: the process could not start without a database and exited with code 1. After the variable was added a manual deploy succeeded. Both deploys are visible in the history, which doubles as evidence of the exit-code behaviour described in Section 8.

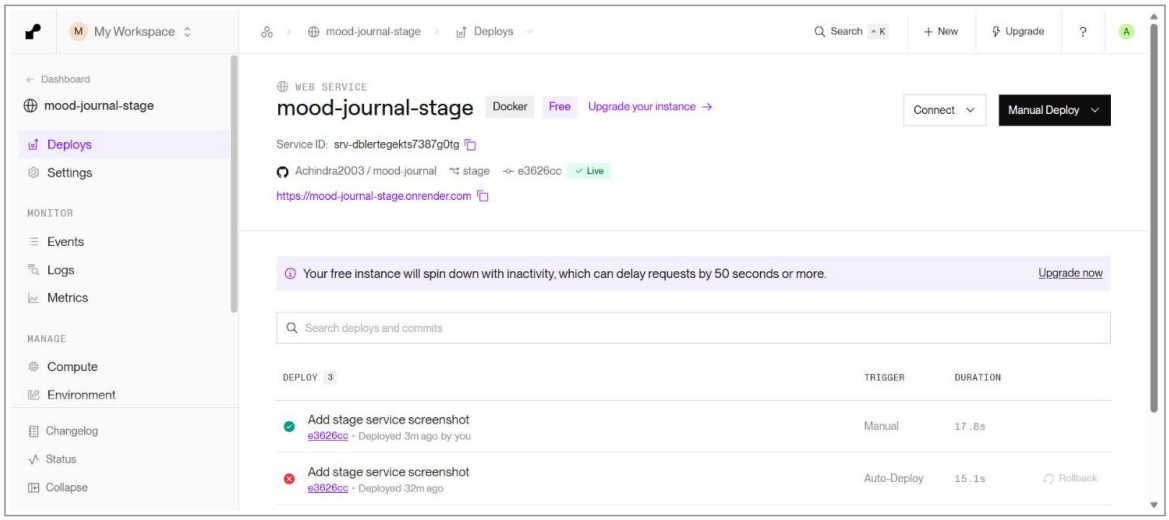


Figure 8 — Stage deploy history: the auto-deploy failed (no DATABASE_URL), the manual deploy afterwards is Live.

The staging site was then exercised end to end: an entry was added and it is stored in moods_stage. The list also shows the entry from the earlier local run, because the local environment file pointed at the same staging database.

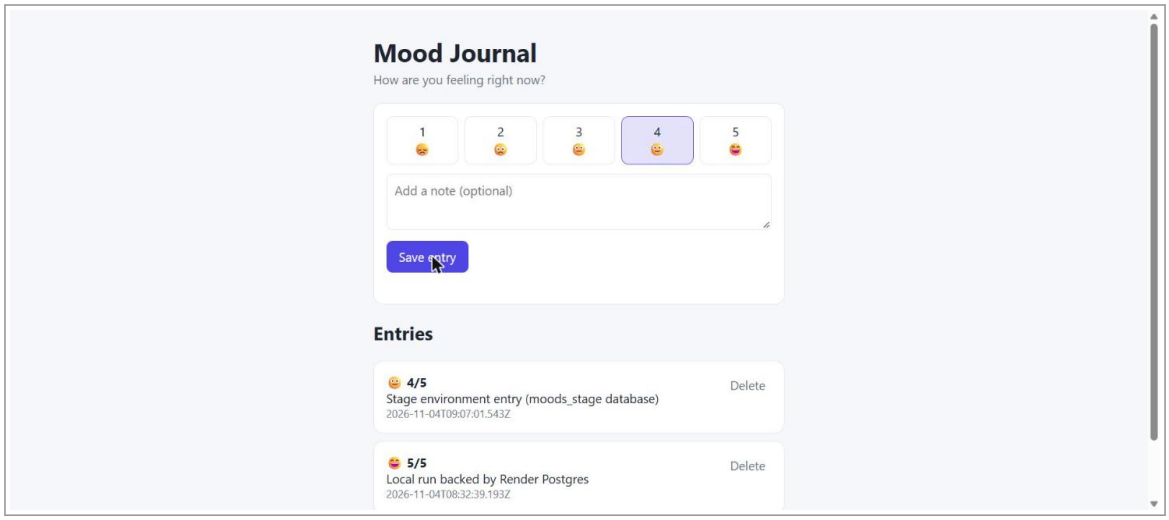


Figure 9 — Staging site mood-journal-stage.onrender.com with entries stored in the moods_stage database.

7. Merge to Production & Persistence Proof

The production service needed its own `DATABASE_URL` (pointing at moods_prod) before the Postgres code reached it; otherwise the merged build would crash at startup. The variable was added to the dashboard first (names only are visible on this page, values are masked).

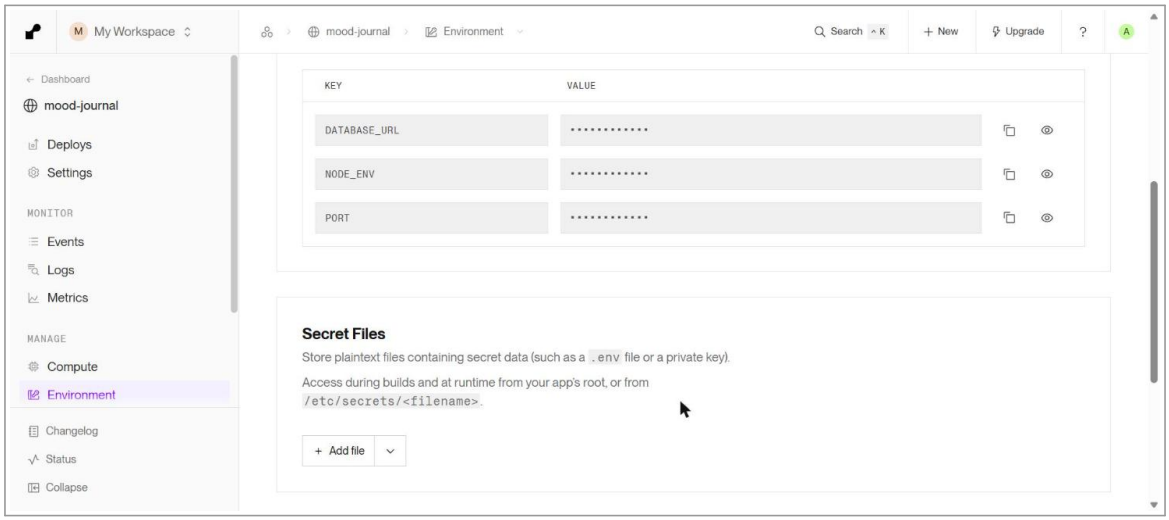


Figure 10 — Production service environment: DATABASE_URL, NODE_ENV and PORT are set (values masked).

A pull request from `stage` to `main` was opened. Both CI runs passed (push and pull_request, 29 s each) and the PR was merged with a merge commit.

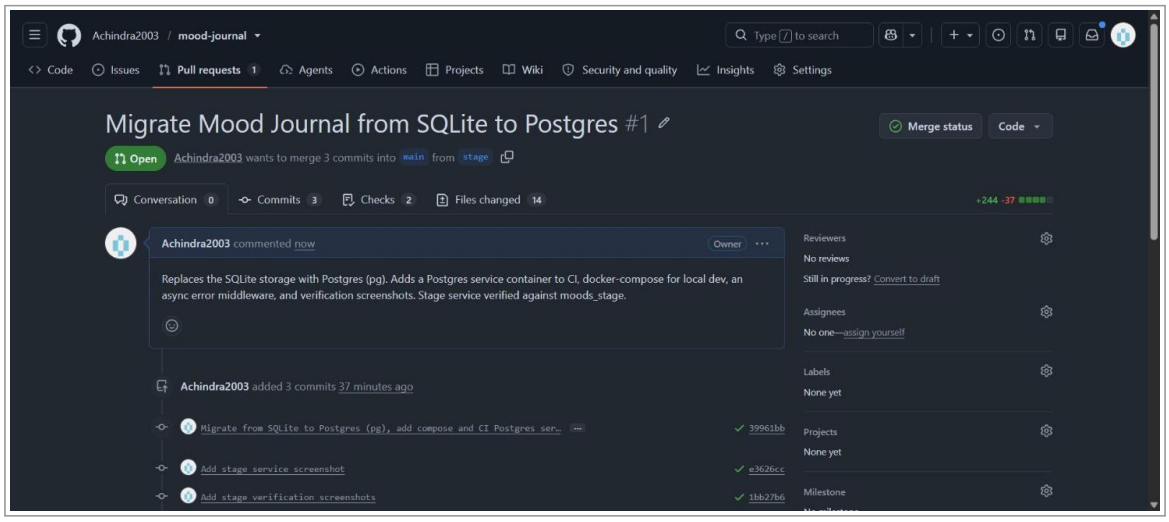


Figure 11 — Pull request #1 (stage → main) open with passing checks.

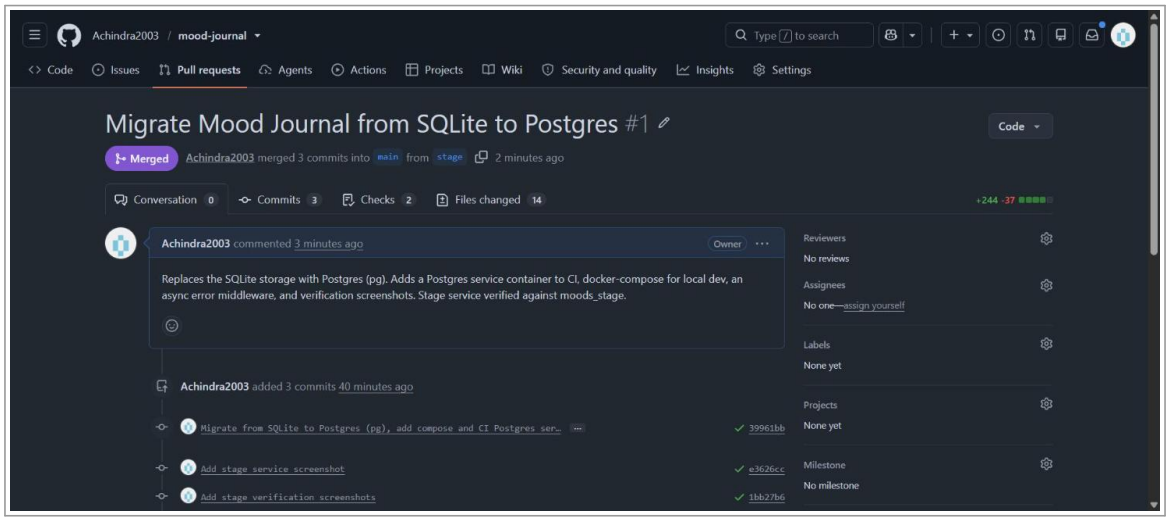


Figure 12 — Pull request #1 merged into main (merge commit 51f99c6).

The merge triggered an automatic production deploy of commit `51f99c6`, which went live in 36 seconds. The production health endpoint then reported the database as connected.

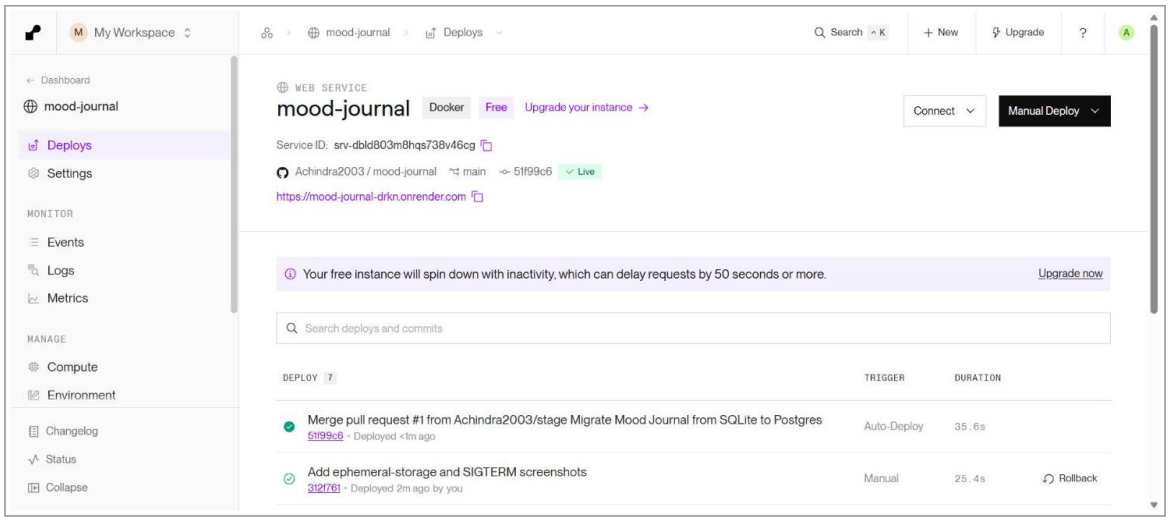


Figure 13 — Production deploy history: the merge commit 51f99c6 deployed automatically and is Live.

```
$ curl https://mood-journal-drkn.onrender.com/health
{"status":"UP","database":"CONNECTED","application":"Mood Journal","uptime":16,...}
```

An entry was added on production, and then the **same redeploy that erased the SQLite data** was repeated. This time the entry survived, because it lives in the managed database and not in the container.

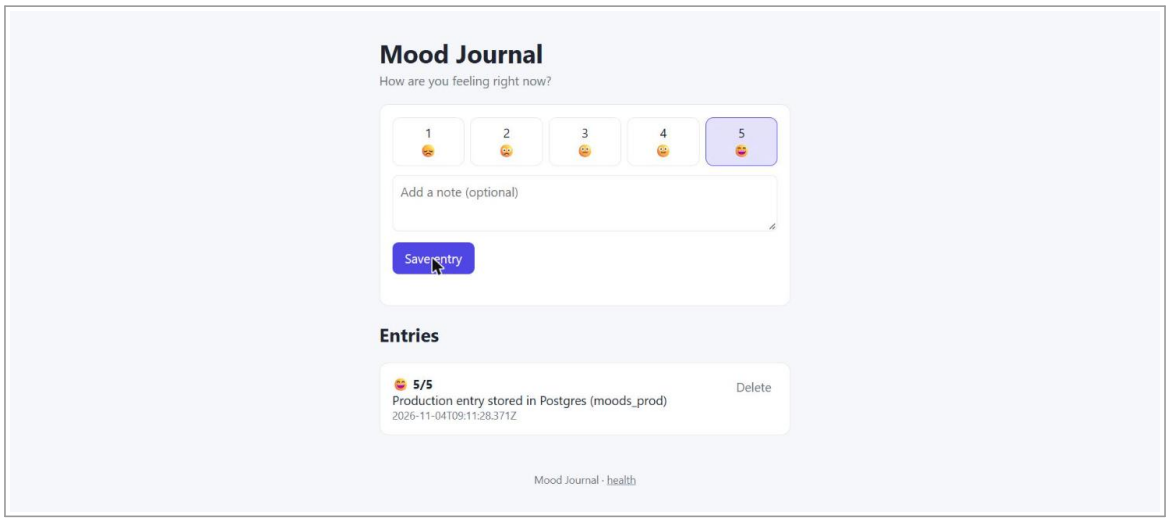


Figure 14 — Production site after the merge: an entry stored in PostgreSQL (moods_prod).

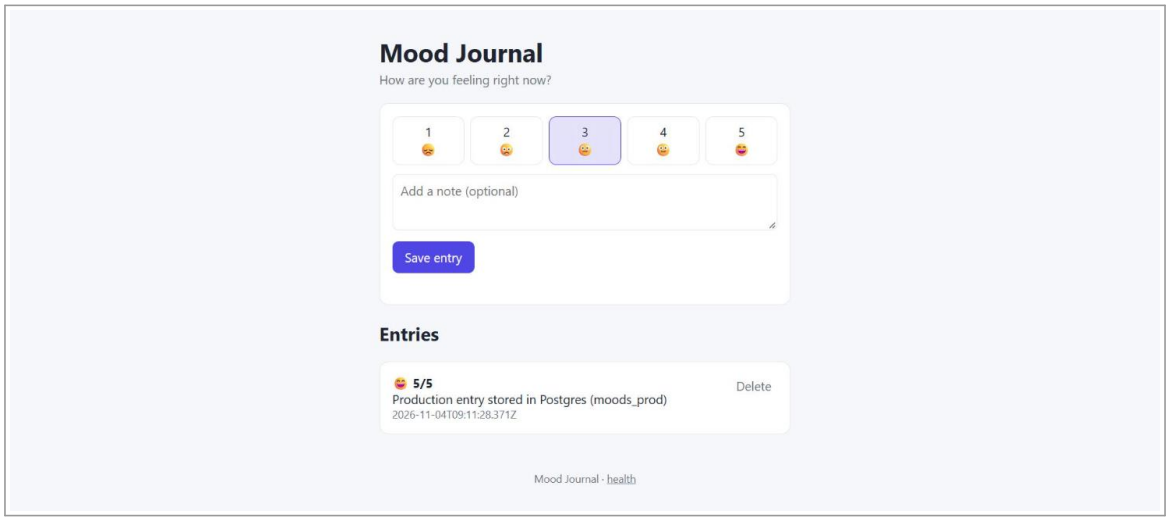


Figure 15 — The same site after redeploying commit 51f99c6: the entry is still present.

```
$ render deploys create srv-dbld803m8hqs738v46cg
Live   51f99c6   api   2026-11-04T09:11:56Z -> 09:12:19Z

$ curl https://mood-journal-drkn.onrender.com/api/moods
[{"id":1,"mood":5,"note":"Production entry stored in Postgres (moods_prod)",...}]
```

8. Process Lifecycle: Signals & Exit Codes

A process that serves requests and holds a database pool should stop in an orderly way. The server installs handlers for `SIGTERM` (sent by Render on every deploy and restart, and by `docker stop`) and `SIGINT` (Ctrl+C). On either signal it stops accepting new connections, closes the HTTP server, closes the database pool, and then exits. A flag prevents the handler from running twice, and a 10 second timer forces exit code 1 if open connections keep the server from closing.

```
process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));
process.on('SIGINT',  () => gracefulShutdown('SIGINT'));

server.close(async (error) => {          // stop accepting requests
  ...; await closeDatabase();            // release the pool
  process.exit();                        // 0 unless process.exitCode was set to 1
});
```

The Dockerfile starts the process with `CMD ["node", "server.js"]` (exec form). Node therefore runs as PID 1 and receives `SIGTERM` directly; starting it through `npm start` or a shell would swallow the signal.

SIGTERM on Render. Every redeploy of the production service shows the old instance receiving the signal and closing cleanly, followed by the new instance going live.

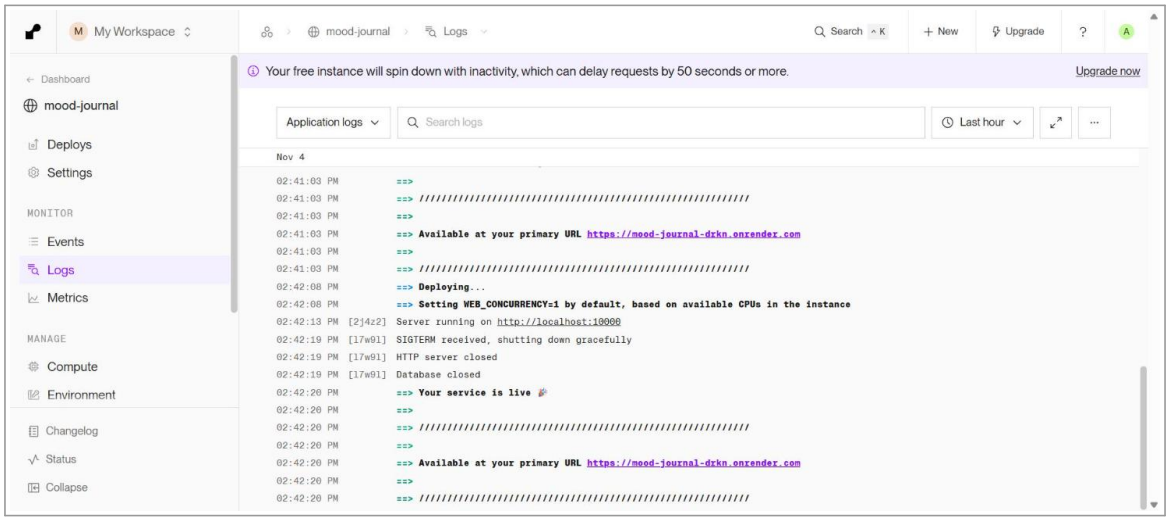


Figure 16 — Render logs of mood-journal: SIGTERM received, HTTP server closed, Database closed, then the new instance is live.

SIGINT locally. The same handler runs when the server is stopped with Ctrl+C.

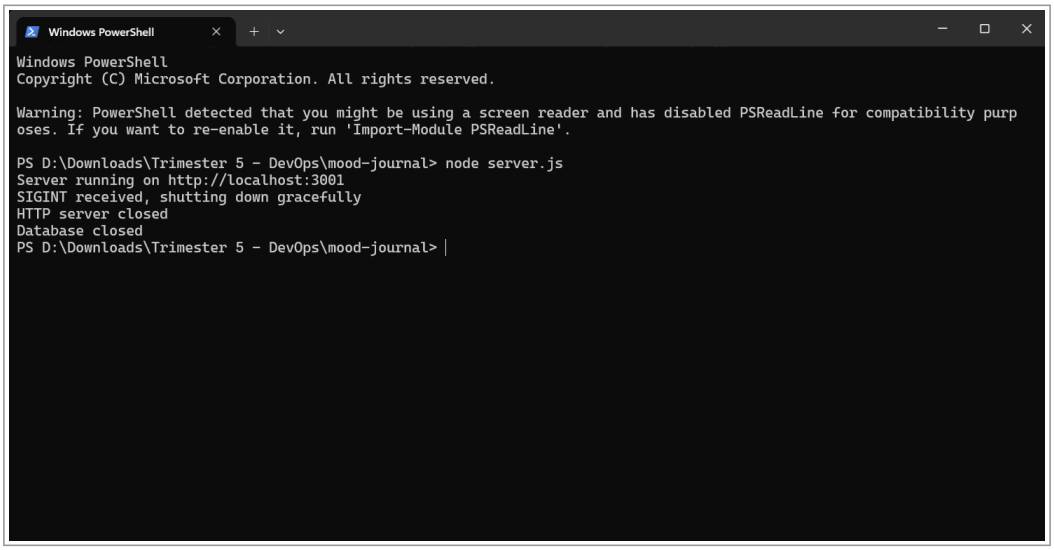


Figure 17 — Local run (PowerShell): Ctrl+C triggers SIGINT, the server and database close in order. The exit code was not echoed in this capture.

Exit code 1 on startup failure. With an unreachable database the process logs the error and exits with code 1, which is what lets a platform detect a failed deploy. This was run locally with a deliberately bad connection string.

```
PS> $env:DATABASE_URL = "postgres://bad:bad@localhost:1/x"
PS> node server.js
Failed to initialize database: AggregateError [ECONNREFUSED]
PS> echo "exit code: $LASTEXITCODE"
exit code: 1
```

Scenario	Signal / trigger	Observed behaviour	Exit code
Render deploy or restart	SIGTERM	Three shutdown log lines, then new instance live	Not shown by Render
Ctrl+C locally	SIGINT	Three shutdown log lines, prompt returns	Not captured in the screenshot

Scenario	Signal / trigger	Observed behaviour	Exit code
Bad DATABASE_URL	none (startup error)	Error logged, process ends	1 (printed locally)
Missing DATABASE_URL on stage	none (startup error)	Deploy marked failed in Render history	non-zero (deploy failed)

Limitation: Windows does not deliver `SIGTERM` to a plain Node process, so that signal was verified on Render, where the service runs on Linux, and not on the local machine.

9. Health Checks

The `/health` endpoint runs `SELECT 1` against the database and returns `200` with `status: UP` and `database: CONNECTED`, or `503` with `DOWN` when the database cannot be reached. It is used in three places: Render's health check path (a deploy only goes live once it answers), the Dockerfile `HEALTHCHECK`, and the CI smoke test of the built container. The path misconfiguration described in Section 3 shows why this matters: a wrong path kept a deploy from ever becoming healthy.

10. Beyond the Baseline: Self-Learning Initiatives

1. **GitHub Actions CI with a real Postgres service container.** Install, test, Docker build and container smoke test on every push and pull request to main and stage.
2. **Automated tests against a real database.** Six tests cover validation, health, create, list and delete, run in CI against PostgreSQL rather than a mock.
3. **Health endpoint wired in three places.** Render health check path, Dockerfile HEALTHCHECK and the CI smoke test all use the same database-aware route.
4. **Graceful shutdown and exit codes.** SIGTERM and SIGINT handlers, a forced-exit timer, and exit code 1 on startup or shutdown failure, with Node running as PID 1.
5. **Hardened container.** Non-root user, production dependencies only, exec-form CMD and a writable-directory fix found through a failing CI run.
6. **Environment separation.** Separate databases and services for stage and production, with secrets kept in the Render dashboard and out of the repository.
7. **Ephemeral-versus-managed storage proof.** The same redeploy erased SQLite data and left PostgreSQL data intact, demonstrated on the same application.

11. Challenges Encountered & How They Were Resolved

Challenge	Cause	Resolution
First CI run failed	Non-root container user could not create the SQLite file in /app	Created /app/data owned by the node user; CI #2 passed
First Render deploy never went healthy	Git Bash rewrote /health into C:/Program Files/Git/health	Corrected the path from PowerShell, cancelled the stuck deploy, redeployed
Redeploy queued behind a stuck deploy	Render runs one deploy at a time per service	Cancelled the stuck deploy so the queued one could run
Only one free Postgres allowed	Free plan limit per account	Created two databases (moods_prod, moods_stage) inside the existing instance

Challenge	Cause	Resolution
Stage deploy failed on first run	DATABASE_URL secret not yet entered	Added the variable in the dashboard and redeployed
Async errors could crash the process	Express 4 does not catch rejected promises in handlers	Wrapped handlers and added an error-handling middleware

12. Key Learnings & Engineering Reflections

Ephemeral filesystems are a design constraint, not a detail. Seeing the data vanish after an ordinary redeploy made it clear why state belongs in a managed service, and why the same redeploy is harmless once it does.

CI catches the problems that only appear in the container. The tests passed locally while the container failed to start, because file permissions differ for a non-root user. Building and starting the image in CI is what found it.

A process must be a good citizen of its platform. Handling signals, closing resources, returning meaningful exit codes and running as PID 1 are what let Render deploy, replace and diagnose the service without losing requests or hiding failures.

Environments should differ only by configuration. The same image and code ran on stage and production with different database URLs, so promotion by pull request carried no surprises. Secrets stayed in the platform and out of Git.